

PRACTICAL WEEK – 10

Session – 10

TASK 1: Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ cd /mnt/c/JavaPrograms
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ echo "This is an inode test file." > original.txt
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ls -li original.txt
23925373020656157 original.txt
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ stat original.txt
  File: original.txt
  size: 28          Blocks: 0          IO Block: 4096   regular file
Device: 0,69      Inode: 23925373020656157  Links: 2
Access: (0777/-rwxrwxrwx)  Uid: ( 1000/keerthi_priya)   Gid: ( 1000/keerthi_priya)
Access: 2026-09-26 12:40:19.283817000 +0000
Modify: 2026-09-26 12:40:19.283817000 +0000
Change: 2026-09-26 12:40:19.283817000 +0000
 Birth: -
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ln original.txt hardlink.txt
ln: failed to create hard link 'original.txt' => 'hardlink.txt': Already exists
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ls -li original.txt hardlink.txt
23925373020656157 hardlink.txt 23925373020656157 original.txt
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ stat original.txt
  File: original.txt
  size: 28          Blocks: 0          IO Block: 4096   regular file
Device: 0,69      Inode: 23925373020656157  Links: 2
Access: (0777/-rwxrwxrwx)  Uid: ( 1000/keerthi_priya)   Gid: ( 1000/keerthi_priya)
Access: 2026-09-26 12:40:19.283817000 +0000
Modify: 2026-09-26 12:40:19.283817000 +0000
Change: 2026-09-26 12:40:19.283817000 +0000
 Birth: -
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ln -s original.txt symlink.txt
ln: Already exists
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ls -li original.txt hardlink.txt symlink.txt
23925373020656157 hardlink.txt 27866022694503005 symlink.txt
23925373020656157 original.txt
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ls -l symlink.txt
lrwxrwxrwx 1 keerthi_priya keerthi_priya 12 Sep 22 03:48 symlink.txt -> original.txt
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ls -li original.txt
23925373020656157 original.txt
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ find . -inum 23925373020656157
./hardlink.txt
./original.txt
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |
```

TASK 2: Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and Implementation complexity.

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <time.h>

#define FILE_SIZE 4096

double get_time()
{
    struct timespec ts;

    clock_gettime(CLOCK_MONOTONIC, &ts);

    return ts.tv_sec + ts.tv_nsec / 1000000000.0;
}

/* Memory-mapped I/O */
double mmap_copy(const char *source, const char *destination)
{
    int src_fd, dest_fd;
    struct stat st;
    char *src_map;
    char *dest_map;
    double start, end;

    src_fd = open(source, O_RDONLY);

    if (src_fd == -1)
    {
        perror("open source");
        exit(1);
    }

    if (fstat(src_fd, &st) == -1)
    {
        perror("fstat");
        close(src_fd);
        exit(1);
    }

    dest_fd = open(destination,
                    O_RDWR | O_CREAT | O_TRUNC,
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1

    dest_fd = open(destination,
                    O_RDWR | O_CREAT | O_TRUNC,
                    0644);

    if (dest_fd == -1)
    {
        perror("open destination");
        close(src_fd);
        exit(1);
    }

    if (ftruncate(dest_fd, st.st_size) == -1)
    {
        perror("ftruncate");
        close(src_fd);
        close(dest_fd);
        exit(1);
    }

    start = get_time();

    src_map = mmap(NULL, st.st_size,
                    PROT_READ, MAP_PRIVATE,
                    src_fd, 0);

    if (src_map == MAP_FAILED)
    {
        perror("mmap source");
        exit(1);
    }

    dest_map = mmap(NULL, st.st_size,
                    PROT_READ | PROT_WRITE,
                    MAP_SHARED,
                    dest_fd, 0);

    if (dest_map == MAP_FAILED)
    {
        perror("mmap destination");
        munmap(src_map, st.st_size);
        exit(1);
    }

    memcpy(dest_map, src_map, st.st_size);
```

```

keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1

memcpy(dest_map, src_map, st.st_size);

msync(dest_map, st.st_size, MS_SYNC);

end = get_time();

munmap(src_map, st.st_size);
munmap(dest_map, st.st_size);

close(src_fd);
close(dest_fd);

return end - start;
}

/* Traditional read/write I/O */
double read_write_copy(const char *source, const char *destination)
{
    int src_fd, dest_fd;
    char buffer[4096];
    ssize_t bytes_read;
    double start, end;

    src_fd = open(source, O_RDONLY);

    if (src_fd == -1)
    {
        perror("open source");
        exit(1);
    }

    dest_fd = open(destination,
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (dest_fd == -1)
    {
        perror("open destination");
        close(src_fd);
        exit(1);
    }

    start = get_time();

    while ((bytes_read = read(src_fd,

```

```

keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1
        start = get_time();

        while ((bytes_read = read(src_fd,
                                buffer,
                                sizeof(buffer))) > 0)
        {
            if (write(dest_fd, buffer, bytes_read) != bytes_read)
            {
                perror("write");
                exit(1);
            }
        }

        end = get_time();

        close(src_fd);
        close(dest_fd);

        return end - start;
    }

int main()
{
    const char *source = "input.txt";
    const char *mmap_file = "mmap_copy.txt";
    const char *rw_file = "read_write_copy.txt";

    double mmap_time;
    double rw_time;

    printf("Session 10 - Memory Mapped I/O\n");
    printf("-----\n");

    mmap_time = mmap_copy(source, mmap_file);
    rw_time = read_write_copy(source, rw_file);

    printf("\nMemory-Mapped I/O:\n");
    printf("Using mmap(), memcpy(), msync(), munmap()\n");
    printf("Time: %.9f seconds\n", mmap_time);

    printf("\nTraditional I/O:\n");
    printf("Using read(), write()\n");
    printf("Time: %.9f seconds\n", rw_time);

    printf("\nFiles created successfully:\n");
    printf("1. %s\n", mmap_file);
    printf("2. %s\n", rw_file);

    return 0;
}

```

```
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano session10_task2.c
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ echo "This file is used to demonstrate mmap file I/O." > input.txt
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ cat input.txt
This file is used to demonstrate mmap file I/O.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session10_task2.c -o session10_task2
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session10_task2
Session 10 - Memory Mapped I/O
-----

Memory-Mapped I/O:
Using mmap(), memcpy(), msync(), munmap()
Time: 0.016612961 seconds

Traditional I/O:
Using read(), write()
Time: 0.008751085 seconds

Files created successfully:
1. mmap_copy.txt
2. read_write_copy.txt
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ cat mmap_copy.txt
This file is used to demonstrate mmap file I/O.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ cat read_write_copy.txt
This file is used to demonstrate mmap file I/O.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ cmp mmap_copy.txt read_write_copy.txt
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |
```